

Práctica 1

Introducción al Arduino y al mecanismo PWM para modular una señal digital (LED)

Grado en Ingeniería en Robótica Software

GSyC, Universidad Rey Juan Carlos



©2024 Julio Vega Pérez

©2025 Esther Aguado

Algunos derechos reservados.

Este trabajo se distribuye bajo la licencia CC-BY-SA 4.0.

1. Introducción

El mundo se está automatizando, con grandes cantidades de datos producidos y procesados para fines analíticos, de control y de conexión. Plataformas como Arduino pueden proporcionar una amplia gama de automatización y procesamiento de datos si se explota todo su potencial. Esta pequeña placa ofrece amplias funcionalidades y oportunidades para cambiar el mundo que nos rodea. Esta práctica es el primer paso para hacerlo...

Dada la gran capacidad de esta placa para interactuar con el mundo exterior, las principales aplicaciones que se pueden desarrollar con ellos son aquellas en las que se utilizan los diferentes sensores disponibles y se convierten los datos que generan en información útil para analizar y controlar los dispositivos que vamos a experimentar durante el curso.

Antes de poder desarrollar estas aplicaciones, es necesario instalar correctamente el *Entorno de Desarrollo Integrado (IDE)* de Arduino. Ver instrucciones de instalación de Arduino (versión 2) en el foro general de Aula Virtual.

2. Descripción de la placa

Lo primero de todo es identificar los diferentes conectores y elementos de la placa mostrada en la Figura 1.

- **Microcontrolador:** El cuadrado negro en el centro de la placa corresponde al *ATmega2560*, que es el corazón del Arduino Mega. Este microcontrolador se encarga de ejecutar el programa cargado desde el entorno de desarrollo y de gestionar todas las entradas y salidas digitales y analógicas.
- **Pines de Analógicos:** Estos 16 pines, marcados por la inicial A, convierten una tensión de entrada entre 0 y 5 V en un valor entero entre 0 y 1023 mediante un convertidor analógico-digital (ADC). Se utilizan para leer sensores que devuelven valores continuos, como potenciómetros o sensores de luz. Para generar una salida analógica, se emplea una señal de modulación por ancho de pulso (PWM), que veremos en detalle en esta práctica. Los pines PWM se pueden reconocer por el símbolo ~.
- **Pines de Ditaless:** Estos 54 pines, numerados con la letra D, solo tienen dos estados posibles: alto (HIGH) o bajo (LOW). Se utilizan para encender o apagar dispositivos y para leer estados de sensores simples, como botones o interruptores.

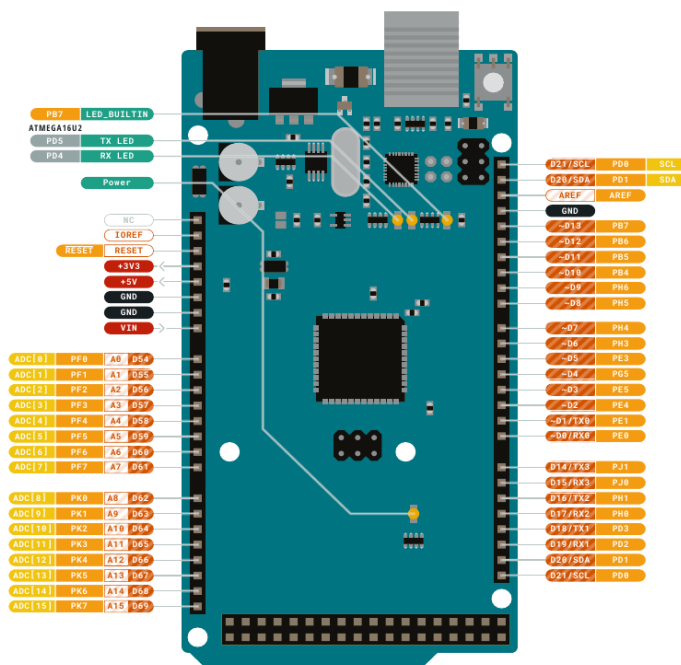


Figura 1: Diagrama de pines placa Arduino Mega 2560 R3.

Si observas la Figura, verás que algunos pines digitales también pueden funcionar como analógicos según cómo se configuren (pines A0–D54 hasta A15–D69). Otros pines se pueden configurar como *PWM* (por ejemplo, pines ~D0 a ~D13) y algunos se comparten con puertos de comunicación (por ejemplo, pines D14–D21), entre otras funcionalidades.

- Alimentación: La placa puede encenderse conectándola al ordenador por USB o usando una fuente externa en el pin VIN o en el conector jack de alimentación.
 - 3.3V y 5V: Pines que proporcionan tensión regulada directamente desde la placa. En la placa utilizada, la alimentación y los niveles lógicos de entradas y salidas funcionan a *5V por defecto*.
 - GND: Pines de tierra comunes para cerrar el circuito eléctrico. Todos los módulos deben conectarse a GND para compartir la misma referencia.
 - VIN: Permite conectar una entrada de tensión externa. Se recomiendan valores entre 7–12 V. El regulador interno convierte esta tensión a 5V.
 - Conector USB: A través de este puerto la placa puede recibir alimentación y, además, establecer comunicación con el ordenador para cargar programas o intercambiar datos mediante el puerto serie.
- LEDs integrados: En la esquina superior izquierda de la Figura 1 aparecen marcados en verde cuatro LEDs:
 - LED Built In (PB7), utilizado para pruebas de encendido o para ejecutar programas de ejemplo.
 - LED TX (PD5) y RX (PD4), que indican la actividad de transmisión y recepción de datos, respectivamente, a través del puerto serial.
 - LED de encendido: muestra el estado general de la placa.
- Comunicación: Arduino puede comunicarse con otros dispositivos de varias formas:

- Comunicación serial(UART): usa los pines *TX* y *RX* para enviar y recibir datos, por ejemplo con otras placas, módulos Bluetooth o GPS.
- I²C: con solo dos cables, los pines *SDA* (datos) y *SCL* (reloj) permiten conectar varios sensores o pantallas al mismo tiempo. Ideal para proyectos donde varios dispositivos necesitan hablar con Arduino al mismo tiempo.
- SPI: mediante el conector ICSP situado bajo el microcontrolador, Es rápido, y se usa con tarjetas SD, pantallas o sensores que requieren gran velocidad.

3. Introducción a la técnica PWM

Para empezar a trastear con la placa vamos a aprender a conectar y controlar un diodo emisor de luz (LED) usando el Arduino. Lo primero que hemos de tener en cuenta es que todos los pines digitales de Arduino funcionan, como su nombre indica, de manera digital. Es decir, pueden entregar solo dos estados: encendido (*HIGH*, 5V) o apagado (*LOW*, 0V). Si conectamos un LED directamente a un pin digital, únicamente podríamos encenderlo o apagarlo, sin posibilidad de variar su brillo.

Para poder controlar la intensidad del LED, necesitamos un mecanismo especial llamado PWM (Pulse Width Modulation, o Modulación por Ancho de Pulso). Este sistema nos permite simular señales analógicas a partir de señales digitales.

En el caso de Arduino, la placa trabaja con una tensión de 5V. Si queremos simular una tensión intermedia (por ejemplo, 2.5V), el pin debe estar encendido la mitad del tiempo y apagado la otra mitad. A esto lo llamamos ciclo de trabajo (*duty cycle*). Así:

- Un ciclo de trabajo del 100 % equivale a salida continua de 5V (LED al máximo brillo).
- Un ciclo de trabajo del 50 % significa que el pin está encendido el 50 % del tiempo y apagado el otro 50 % (LED con brillo medio).
- Un ciclo de trabajo del 25 % implicaría que el pin está encendido solo un cuarto del tiempo (LED con brillo bajo).

De esta manera, variando el ciclo de trabajo podemos controlar la intensidad percibida del LED. Arduino nos facilita este proceso con la función `analogWrite()`, que envía una señal PWM de 8 bits de resolución (valores de 0 a 255).

4. Montaje del LED

Pasamos a instalar y usar el LED. Lo primero que debemos tener en cuenta es la polaridad de un LED. Si nos fijamos con el en la mano, vemos que hay un pin más corto que otro (Figura 2), y que además, en ese mismo lado el encapsulado del LED presenta una muesca o marca plana. Pues bien, este lado es el negativo (cátodo); por el contrario, el pin más largo (y sin muesca) es el ánodo o polo positivo.

Recordemos que el ánodo y cátodo de un dispositivo depende de si el dispositivo en cuestión consume o aporta energía. Siendo el ánodo el electrodo por el cual entra la energía al dispositivo; y el cátodo, por el que sale. Así, en una batería, en modo *aportar energía* (no en modo cargando), la energía sale del positivo y entra en el negativo; es por ello que el ánodo es el negativo y, el cátodo, el positivo. Por el contrario, en un diodo LED emisor de luz, que consume energía, la corriente entra por el positivo y sale por el negativo; por tanto, el ánodo es el positivo y, el cátodo, el negativo.

En la Figura 3 vemos cómo quedaría la conexión del LED con la placa de Arduino. Como se puede apreciar, estamos utilizando el pin 9, marcado como $\sim D9$, como salida de control para el LED, y la conexión a

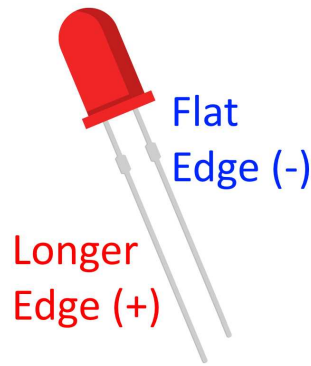


Figura 2: Polaridad de un LED.

GND como toma de tierra. Se debe utilizar una resistencia para proteger el LED y evitar que se dañe por exceso de corriente. En este caso hemos utilizado una de $220\ \Omega$ ¹.

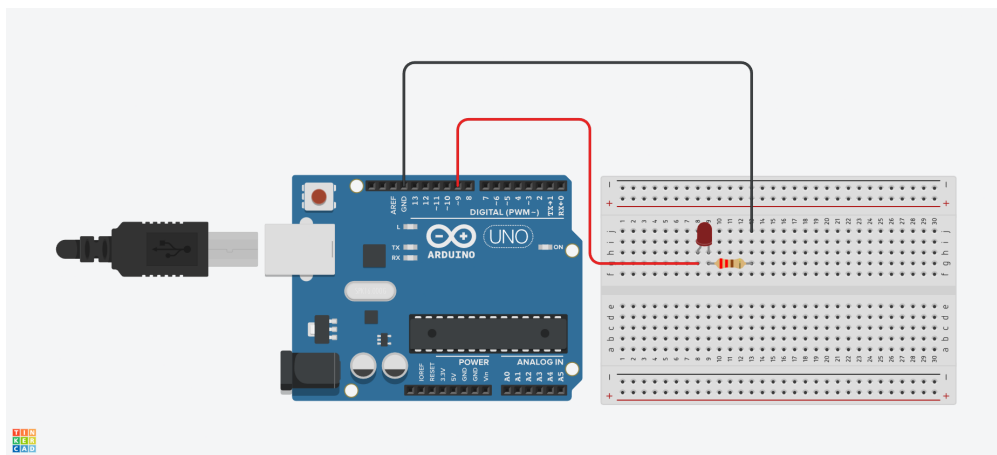


Figura 3: Conexión de un LED con Arduino.

5. Manejo básico del LED con PWM

La programación en Arduino se realiza mediante el IDE que acabamos de instalar, que permite escribir código en un lenguaje basado en C++. Todo programa de Arduino, llamado *sketch*, tiene al menos dos funciones principales:

- **setup()**: Se ejecuta una sola vez al inicio del programa. Aquí se configuran los pines, inicializan variables y se preparan los dispositivos para su funcionamiento.
- **loop()**: Se ejecuta de manera continua después de la inicialización. Aquí se colocan las instrucciones que deben repetirse durante la operación del dispositivo.

Además podemos utilizar comentarios, macros, definir nuestras propias variables, crear bucles, funciones, y otras estructuras de programación.

¹La resistencia limitadora se calcula con la Ley de Ohm: $R = \frac{V_{fuente} - V_{LED}}{I}$. Para un LED típico (2V, 20mA) y una salida de 5V de Arduino, el valor recomendado está entre $150\ \Omega$ y $330\ \Omega$. Elegir un valor dentro de este rango no es muy crítico: valores más altos solo reducirán ligeramente el brillo del LED, mientras que valores más bajos aumentarán la corriente pero siempre dentro de límites seguros para LEDs estándar.

- Comentarios: Se escriben con `//` para comentarios de una sola línea o con `/* ... */` para comentarios de varias líneas.
- Constantes: Se pueden declarar variables cuyo valor no puede modificarse después de su inicialización mediante `const`.

Para controlar el LED, debemos indicar qué pin vamos a usar como salida. Normalmente se utiliza una constante con `const int` para definir el número de pin, lo que aporta legibilidad y facilita modificaciones futuras.

```
1  const int ledPin = 9;  // Pin digital con PWM
2
3  void setup() {
4      pinMode(ledPin, OUTPUT); // Configura el pin como salida
5  }
```

A continuación, podemos controlar el LED de dos formas:

5.1. Encendido o apagado simple con `digitalWrite()`

Esta función permite encender o apagar el LED enviando una señal digital (HIGH o LOW) al pin.

```
1  void loop() {
2      digitalWrite(ledPin, HIGH); // Enciende el LED
3      delay(1000);                // Espera 1 segundo
4      digitalWrite(ledPin, LOW);  // Apaga el LED
5      delay(1000);                // Espera 1 segundo
6  }
```

5.2. Control de brillo con `analogWrite()` (PWM)

Esta función permite variar el brillo del LED mediante PWM. El parámetro que pasamos a `analogWrite()` determina el ciclo de trabajo de la señal.

```
1  void loop() {
2      analogWrite(ledPin, 127); // Brillo medio
3      delay(1000);
4      analogWrite(ledPin, 255); // Brillo máximo
5      delay(1000);
6      analogWrite(ledPin, 64);  // Brillo bajo
7      delay(1000);
8  }
```

6. Ejercicio

Modifica el programa anterior para que la iluminación del LED aumente y disminuya de forma gradual, creando un efecto de desvanecimiento. Para lograrlo, utiliza bucles `for` que incrementen y decrementsen el valor de brillo y, si es necesario, disminuye el tiempo de espera para que el cambio de intensidad sea más suave y continuo.

Entrega de la práctica

La entrega se realizará en el repositorio individual de GitHub Classroom. Cada repositorio debe contener al menos el código producido así como un archivo `README.md` que incluya la memoria de la práctica con los siguientes elementos:

- Nombre de los estudiantes.
- Objetivo de la práctica.
- Resumen del trabajo realizado.
- Particularidades: Problemas encontrados, funcionalidad extra, etc.
- Vídeo del funcionamiento del ejercicio.
- Esquemático del circuito (Fritzing, Tinkercad, etc.).